

Hard

Input: $s = \text{"OUZODYXAZV"} , t = \text{"XYZ"}$

Output: "YXAZ"

Ideas:

$\text{mapp} = \{ 'X': 1, 'Y': 1, 'Z': 1 \}$

The substring must contain these characters with their frequencies. It can contain other characters.

We want the smallest substring

$s = \text{"OUZODYXAZV"} , t = \text{"XYZ"}$

$\text{mapp} = \{$

$'X': 0$ is $s[j]$ in mapp ?

$'Y': 0$ decrease frequency

$'Z': 0$ when all the frequencies are 0, it means we found a valid substring.

$\}$

We could have a min-substring that keeps the minimum we've found so far.

The current length is: $j-i+1$

1st substring: 7

When we find a substring, we shrink the window from the left checking if it's still valid.

When i at index 2, we found a smaller substring w/ length 5.

min-substring found so far: $\text{"ZODYX"} \rightarrow \text{length } 5$

When i points to a character in the map we increase the frequency of that specific character, in the next iteration the substring would no longer be valid, because the sum of the values in the map would be different than 0

The j iterator would now find another $'Z'$ at index 8, making the substring valid again. At this point, we shrink from the left again

We would find the smallest substring: $\text{"YXAZ"} \rightarrow \text{length } 4$

Pseudo Code

Create HashMap { Key: character; Value: char's frequency } of string t .

$i = 0$; min-length = 0;

map-count = sum of values in HashMap

Loop j over string s :

Loop while map-count is 0:

Compare current length with minimum length so far

Is character at i in the hash map?

Increase character frequency

Increase total map count

Shrink window

Save substring

Else:

shrink window

Is character at j in the hash map?

Reduce frequency of character at j

Reduce total map count

Compare current length with minimum length so far

Return substring

Solution

The solution I showed and came up with is not 100% right. It had some mistakes but I'd say I got 80% right, I didn't catch some details.

A different solution proposes the idea of having 2 hash maps.

One for the characters we need and one for the ones we have.

We also keep track of the sum of the values in both the hashmaps and compare their values

A substring is only valid if the sum of the characters we have is equal to the ones we need.

This is similar to what I did with map_count.

When we find a valid substring, we remove the left-most character until the substring is no longer valid.

The key idea of this solution is having 2 HashMaps to compare the frequency of the characters we need to the ones we have. We join this with the sliding window algorithm to solve it in $O(n)$ time complexity.

Code:

```
countMap, window = {}, {}
```

```
for c in t:
```

```
    countMap[c] = countMap.get(c, 0) + 1
```

```
have, need = 0, len(countMap)
```

```
i, res, resLen = 0, [-1, -1], float('inf')
```

```
for j in range(len(s)):
```

```
    c = s[j]
```

```
    window[c] = window.get(c, 0) + 1
```

```
    if c in mapCount and window[c] == mapCount[c]:
```

```
        have += 1
```

```
    while have == need:
```

```
        if (j - i + 1) < resLen:
```

```
            res = [i, j]
```

```
            resLen = j - i + 1
```

```
        window[s[i]] -= 1
```

```
        if s[i] in mapCount and window[s[i]] < mapCount[s[i]]:
```

```
            have -= 1
```

```
        i += 1
```

```
i, y = res
```

```
return s[i:y+1] if reslen != float('inf') else ""
```

My solution with some corrections:

```
mapp = {}
```

```
for c in t:
```

```
    mapp[c] = mapp.get(c, 0) + 1
```

```
i, min_length = 0, float('inf')
```

```
map_count = sum(mapp.values())
```

```
substring = ""
```

```
for y in range(len(s)):
```

```
    if s[y] in mapp
```

```
        if mapp[s[y]] > 0:
```

```
            map_count -= 1
```

```
            mapp[s[y]] -= 1
```

```
        while map_count == 0
```

```
            if (y - i + 1) <= min_length:
```

```
                substring = s[i:y+1]
```

```
                min_length = y - i + 1
```

```
            if s[i] in mapp:
```

```
                mapp[s[i]] += 1
```

```
            if mapp[s[i]] > 0:
```

```
                map_count += 1
```

```
            i += 1
```

```
return substring
```

Time complexity: $O(n+m)$

Space complexity: $O(K)$